



UNIVERSIDAD TÉCNICA DE AMBATO

Carrera de Ingeniería en Software

Asignatura: Algoritmos de Programación

PROYECTO FORMATIVO INTEGRADOR

Desarrollo de un aplicativo web didáctico para la retroalimentación del aprendizaje de Lógica y Algoritmos de Programación mediante ejercicios aplicados a problemas de la vida real

Docente	José Rubén Caiza C.
Asignatura	Algoritmos de Programación
Proyecto	Aplicativo web didáctico de lógica y algoritmos
Curso / paralelo	1 Semestre Ingeniería en Software “B”
Estudiantes	Guaman Chanahuano Hamilton Alexander Josué Alejandro Villacres Toalombo Monar Parco Jhair Alexander Muyulema Moyolema Mateo Alexander
Periodo académico	2026
Lugar y fecha	Ambato – Ecuador, Junio de 2026
URL del sistema	https://logicweb.pythonanywhere.com
Link Repositorio GitHub	https://github.com/mateomuyulema2007-cmyk/Proyecto-Final

LogicWeb UTA

Aplicativo didactico de logica, algoritmos y retroalimentacion.

LogicWeb UTA

Aprender lógica programando soluciones reales



Índice de contenido

Índice de contenido	2
1. Resumen Ejecutivo	5
2. Introducción.....	6
3. Tema del Proyecto	7
4. Título del Proyecto.....	7
5. Planteamiento del Problema	7
6. Justificación	8
7. Objetivos	9
7.1 Objetivo General	9
7.2 Objetivos Específicos.....	9
8. Alcance del Proyecto.....	10
9. Marco Teórico.....	10
9.1 Lógica de Programación.....	10
9.2 Algoritmos	11
9.3 Variables, Tipos de Datos y Operadores	11
9.4 Estructuras Condicionales.....	11
9.5 Ciclos: for y while	11
9.6 Contadores y Acumuladores	11
9.7 Funciones	11
9.8 Programación Orientada a Objetos (POO)	12
9.9 Python y Flask	12
9.10 SQLite	12
9.11 HTML5, CSS3 y JavaScript.....	12
9.12 PythonAnywhere	13
9.13 Aplicativos Web Educativos	13
9.14 Patrón MVC (Modelo-Vista-Controlador).....	13
9.15 Metodología Iterativa Incremental	14
10. Metodología de Desarrollo	14
10.1 Fase de Análisis.....	14
10.2 Patrón MVC aplicado al proyecto	15
10.3 División de trabajo del equipo	15
10.4 Fases del ciclo iterativo	15
10.5 Fase de Diseño	16
10.6 Fase de Implementación	16
10.7 Fase de Pruebas	16
10.8 Fase de Despliegue	16
11. Requerimientos del Sistema	18



11.1 Requerimientos Funcionales Implementados.....	18
11.2 Requerimientos No Funcionales Implementados.....	18
12. Arquitectura del Sistema	19
12.1 Arquitectura de Tres Capas	19
12.2 Estructura de Archivos del Proyecto.....	19
12.3 Flujo de Navegación	20
12.4 Mapa de rutas Flask (Capa Controlador).....	21
13. Diseño Funcional del Sistema.....	21
13.1 Módulo de Autenticación.....	21
13.2 Módulo de Inicio (Dashboard).....	22
13.3 Módulos de Contenido Teórico	23
13.4 Módulo de Ejercicios Interactivos.....	27
13.5 Módulo de Progreso.....	28
13.6 Módulo de Laboratorio Lógico.....	29
13.7 Módulo de Notas de Apoyo.....	30
13.8 Panel de Reportes Docente.....	30
14. Diseño de Datos.....	34
14.1 Modelo Entidad-Relación	34
Clase usuarios – gestión de autenticación	34
Clase practicas – historial de ejercicios interactivos y laboratorio	34
Clase promedios – cálculos académicos	34
Clase actividades_modulo – 5 actividades por módulo teórico	35
15. Proceso de Implementación Paso a Paso	36
PASO 1 – Configuración del Entorno de Desarrollo.....	36
PASO 2 – Creación de la Estructura del Proyecto	36
PASO 3 – Diseño e Implementación de la Base de Datos(Cñases)	37
PASO 4 – Implementación del Backend Flask (app.py)	38
PASO 5 – Implementación del Frontend.....	38
PASO 6 – Cinco actividades por módulo	39
PASO 7 – Panel de Reportes Docente	39
PASO 8 – Pruebas en Entorno Local	39
PASO 8 – Preparación para Despliegue en PythonAnywhere	39
PASO 9 – Configuración del Archivo WSGI en PythonAnywhere.....	40
PASO 10 – Inicialización de la Base de Datos en Producción.....	40
PASO 10 – Verificación y Puesta en Producción	40
15.1 Control de versiones y repositorio del proyecto	41
16. Integración de Conocimientos de la Asignatura	43
17. Metodología de desarrollo y división del trabajo	43
17.1 Metodología utilizada.....	43



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



17.2 Planificación del proyecto.....	43
17.3 Sprints o iteraciones realizadas.....	44
17.4 Ceremonias Ágiles Aplicadas.....	44
17.5 División del trabajo entre integrantes	45
18. Tabla de Participaciones.....	45
19. Resultados Obtenidos	46
20. Cronograma de Ejecución	47
21. Recursos Utilizados.....	47
21.1 Recursos Tecnológicos.....	47
21.2 Presupuesto.....	48
22. Riesgos del Proyecto	48
23. Conclusiones.....	48
24. Recomendaciones	49
25. Bibliografía/ Documentación web	49
26. Anexos	50
Anexo A – Rúbrica de Evaluación	50
Anexo B – Propuesta de Ejercicios de Vida Real Implementados.....	50
Anexo C – URL de Acceso al Sistema.....	51



1. Resumen Ejecutivo

El presente informe técnico-académico documenta el proceso de análisis, diseño, desarrollo, implementación, validación y despliegue del aplicativo web educativo **LogicWeb UTA**, elaborado como **proyecto formativo integrador** de la asignatura **Algoritmos de Programación** de la **Universidad Técnica de Ambato**.

La solución propuesta surge como respuesta a la necesidad de fortalecer el aprendizaje de la lógica computacional y del análisis algorítmico en estudiantes que inician su formación en programación. Para ello, se diseñó una plataforma web didáctica orientada al refuerzo progresivo de conocimientos mediante la integración de contenidos teóricos, ejemplos explicativos, ejercicios interactivos contextualizados en situaciones de la vida real, retroalimentación automática y mecanismos de seguimiento del desempeño del usuario. Desde el punto de vista tecnológico, el sistema fue desarrollado con **Python** y el framework **Flask** para la capa de lógica de negocio y backend; **HTML5**, **CSS3** y **JavaScript** para la capa de presentación e interacción del usuario; **SQLite** como sistema gestor de base de datos relacional embebido; y **PythonAnywhere** como entorno de despliegue y publicación en la nube. Como resultado, el aplicativo fue publicado exitosamente y se encuentra disponible en la dirección pública: <https://logicweb.pythonanywhere.com>.

Entre las funcionalidades implementadas destacan: el **registro de usuarios**, el **inicio y cierre de sesión**, la **visualización estructurada de contenidos teóricos**, los **ejercicios interactivos con validación de entradas y generación de retroalimentación inmediata**, el **almacenamiento persistente del historial de prácticas**, y el **seguimiento del progreso académico** mediante indicadores como total de ejercicios realizados, respuestas correctas y porcentaje de efectividad. En términos académicos, LogicWeb UTA constituye una evidencia concreta de integración de conocimientos adquiridos en la asignatura, ya que articula conceptos de lógica de programación, estructuras de control, modularidad, persistencia de datos, diseño de interfaces y despliegue web en un producto funcional y accesible. Asimismo, el proyecto demuestra que es posible desarrollar una herramienta educativa de utilidad real utilizando exclusivamente tecnologías open source, con una orientación pedagógica clara y un enfoque aplicado al contexto universitario.



2. Introducción

La formación inicial en programación requiere que el estudiante construya una base sólida en **pensamiento lógico, análisis de problemas, estructuración de algoritmos y diseño de soluciones computacionales**. Sin embargo, en la práctica educativa es frecuente observar que muchos estudiantes comprenden de manera aislada ciertos conceptos teóricos —como variables, operadores, estructuras condicionales o ciclos—, pero presentan dificultades al momento de integrarlos en la resolución de problemas completos y contextualizados.

Esta brecha entre teoría y práctica afecta el proceso de aprendizaje, ya que limita la capacidad del estudiante para identificar adecuadamente las entradas, definir el procesamiento necesario y establecer las salidas esperadas. Como consecuencia, se producen errores recurrentes en la formulación de algoritmos, en la validación de datos y en la implementación de soluciones funcionales. Además, la ausencia de herramientas de refuerzo accesibles e interactivas reduce las oportunidades de práctica autónoma y de retroalimentación inmediata.

Frente a esta problemática, se planteó el desarrollo de una plataforma web educativa denominada **LogicWeb UTA**, concebida como un recurso de apoyo para la asignatura **Algoritmos de Programación**. El sistema fue diseñado con un enfoque didáctico orientado al aprendizaje progresivo, combinando explicaciones teóricas, ejemplos ilustrativos y ejercicios interactivos basados en situaciones de la vida real. De esta manera, se busca reforzar la comprensión de la lógica de programación y favorecer la aplicación práctica de los contenidos abordados en clase.

El proyecto fue desarrollado utilizando un stack tecnológico completamente **open source**, conformado por **Flask (Python)** para el backend, **HTML5/CSS3/JavaScript** para el frontend y **SQLite** como base de datos. Esta elección tecnológica no solo permitió construir un sistema funcional, modular y de bajo costo, sino que además garantizó su portabilidad, facilidad de mantenimiento y viabilidad de despliegue en la nube a través de **PythonAnywhere**.

En este contexto, el presente informe expone de manera ordenada los fundamentos conceptuales, las decisiones técnicas, la metodología aplicada, la arquitectura del sistema, los resultados obtenidos y las evidencias funcionales del desarrollo de LogicWeb UTA, consolidándolo como una propuesta académica pertinente, viable y alineada con los objetivos formativos de la carrera.



3. Tema del Proyecto

Aplicación de los conocimientos de lógica, análisis algorítmico y programación básica en el desarrollo de un aplicativo web educativo que permita reforzar el aprendizaje mediante contenidos, ejemplos y ejercicios interactivos de la vida real.

4. Título del Proyecto

Desarrollo de un aplicativo web didáctico para la retroalimentación del aprendizaje de Lógica y Algoritmos de Programación mediante ejercicios aplicados a problemas de la vida real.

5. Planteamiento del Problema

En los primeros niveles de la carrera de **Ingeniería en Software**, es frecuente encontrar estudiantes que identifican conceptos fundamentales de programación, tales como variables, operadores, estructuras condicionales, ciclos, contadores y funciones, pero presentan limitaciones al momento de articular dichos elementos dentro de una solución lógica completa. Esta situación evidencia que, en muchos casos, el aprendizaje se centra en la memorización de sintaxis o en la repetición mecánica de ejemplos, sin consolidar un entendimiento profundo del razonamiento algorítmico que sustenta la resolución de problemas.

Esta problemática se agrava cuando el estudiante no dispone de recursos interactivos que faciliten la práctica autónoma, la experimentación y la retroalimentación inmediata. En ausencia de estos apoyos, se observan dificultades en la identificación de entradas, procesos y salidas, en la validación de datos, en la formulación de pasos lógicos y en la resolución de ejercicios contextualizados. Como resultado, el aprendizaje puede volverse fragmentado, poco significativo y escasamente transferible a situaciones reales.

Adicionalmente, los materiales tradicionales de enseñanza suelen enfocarse en la exposición conceptual, dejando en segundo plano la aplicación práctica guiada. Esto limita la posibilidad de que el estudiante comprenda cómo se traducen los conceptos abstractos de lógica y algoritmos en soluciones concretas dentro de un sistema computacional.

Frente a esta realidad, se identificó la necesidad de desarrollar una herramienta académica que integre en un mismo entorno la **teoría**, la **práctica**, la **retroalimentación** y el **seguimiento del desempeño del estudiante**. En respuesta a ello, el proyecto propone y desarrolla **LogicWeb UTA**, un aplicativo web didáctico capaz de presentar contenidos estructurados, permitir la resolución de ejercicios interactivos y almacenar evidencia del progreso del usuario dentro de un entorno accesible, intuitivo y alineado con los objetivos formativos de la asignatura.



6. Justificación

El desarrollo del proyecto **LogicWeb UTA** se justifica desde las dimensiones **pedagógica, técnica, académica y formativa**.

Desde la dimensión pedagógica, el aplicativo contribuye al fortalecimiento del proceso de enseñanza-aprendizaje de la lógica y los algoritmos de programación mediante una estrategia que combina contenidos teóricos, ejemplos explicativos y práctica interactiva. Esta integración favorece una comprensión más significativa de los conceptos, ya que el estudiante no solo accede a información conceptual, sino que también tiene la oportunidad de aplicarla inmediatamente en ejercicios estructurados y contextualizados.

Desde la dimensión técnica, el proyecto permite materializar los conocimientos adquiridos en la asignatura en un producto de software funcional y desplegado en un entorno real. Esto implica aplicar competencias relacionadas con análisis de requerimientos, diseño de interfaces, arquitectura web, programación backend, manejo de bases de datos, validación de formularios, persistencia de información y despliegue de aplicaciones en la nube.

Desde la dimensión académica, LogicWeb UTA representa una evidencia concreta del aprendizaje logrado durante el período formativo, al integrar en una sola solución conceptos fundamentales como variables, condicionales, ciclos, contadores, acumuladores, funciones, modularidad, autenticación de usuarios y almacenamiento persistente de datos. En este sentido, el proyecto supera el nivel de ejercicio aislado y se consolida como una implementación integral con propósito educativo.

Asimismo, el uso de ejercicios basados en situaciones de la vida real —como promedios académicos, cálculo de sueldos, descuentos comerciales, inventarios y consumo de energía— permite que el estudiante reconozca la aplicabilidad de la programación en contextos cotidianos. Esto incrementa la pertinencia del aprendizaje y promueve una relación más directa entre la teoría y su utilidad práctica.

Finalmente, el proyecto se justifica por su viabilidad económica y tecnológica, ya que fue desarrollado íntegramente con herramientas gratuitas y open source, sin costos adicionales de licenciamiento. Esta característica lo convierte en una propuesta replicable, escalable y potencialmente útil como material de consulta, nivelación o apoyo para futuras cohortes estudiantiles.



7. Objetivos

7.1 Objetivo General

Desarrollar e implementar un aplicativo web didáctico que contribuya a la retroalimentación del aprendizaje de Lógica y Algoritmos de Programación mediante contenidos interactivos, ejemplos resueltos y ejercicios aplicados a situaciones de la vida real, desplegado en un servidor de acceso público.

7.2 Objetivos Específicos

- Analizar las necesidades de aprendizaje relacionadas con la asignatura de Lógica y Algoritmos de Programación.
- Diseñar una interfaz web clara, amigable y orientada al uso académico con un tema oscuro moderno.
- Organizar contenidos teóricos y prácticos en seis módulos: Variables y Tipos, Condicionales, Bucles y Ciclos, Contadores, Acumuladores y Funciones.
- Implementar ejercicios interactivos con validación de entradas, procesamiento y retroalimentación de resultados.
- Desarrollar el sistema de autenticación de usuarios (registro, login y logout) con sesiones Flask.
- Almacenar el historial de prácticas y promedios calculados en una base de datos SQLite.
- Desplegar el sistema en PythonAnywhere para garantizar acceso público y permanente.



8. Alcance del Proyecto

El sistema desarrollado y desplegado incluye los siguientes componentes funcionales:

- **Módulo de autenticación:** registro de nuevos usuarios, inicio de sesión y cierre de sesión.
- **Módulo de inicio (dashboard):** página principal de bienvenida con acceso a los módulos del sistema y navegación centralizada.
- **Módulo de contenidos:** presentación de temas teóricos relacionados con la asignatura, organizados de manera estructurada.
- **Módulo de ejercicios interactivos:** actividades contextualizadas en situaciones reales, con cálculo, validación y retroalimentación automática.
- **Módulo de progreso:** visualización del historial de prácticas, métricas de efectividad y resultados almacenados.
- **Módulo de apoyo al aprendizaje:** herramientas complementarias como laboratorio lógico, comparación de lenguajes y notas de apoyo para la exposición.
- **Persistencia de datos:** almacenamiento de usuarios, intentos de práctica, notas retroalimentación y promedios en SQLite.
- **Despliegue en la nube:** sistema accesible públicamente en <https://logicweb.pythonanywhere.com>.

El proyecto no contempla, en su versión actual, la integración con sistemas institucionales externos, inteligencia artificial avanzada, aplicaciones móviles nativas, administración docente avanzada ni despliegue en infraestructura empresarial de alta disponibilidad. Estos componentes se consideran líneas de evolución futura.

9. Marco Teórico

9.1 Lógica de Programación

La lógica de programación constituye la base del razonamiento computacional y permite analizar problemas, descomponerlos en partes y plantear soluciones ordenadas, coherentes y finitas. Su dominio es indispensable para estructurar procedimientos antes de traducirlos a un lenguaje de programación. En el contexto educativo, el desarrollo de la lógica de programación favorece la capacidad de abstracción, el análisis secuencial y la toma de decisiones.



9.2 Algoritmos

Un algoritmo es una secuencia finita, ordenada y precisa de instrucciones orientadas a resolver un problema específico. Todo algoritmo debe poseer claridad, finitud, precisión y una correspondencia lógica entre las **entradas**, el **procesamiento** y las **salidas**. En la formación inicial, la comprensión de algoritmos permite al estudiante construir soluciones antes de enfocarse en la sintaxis de un lenguaje particular.

9.3 Variables, Tipos de Datos y Operadores

Una variable es un espacio de memoria con nombre que almacena un valor que puede cambiar durante la ejecución. Los tipos más comunes en Python son int (enteros), float (decimales), str (cadenas) y bool (verdadero/falso). Los operadores aritméticos, relacionales y lógicos permiten realizar cálculos, comparaciones y tomas de decisiones respectivamente.

9.4 Estructuras Condicionales

Las estructuras condicionales (if, elif, else) permiten que el programa tome decisiones basadas en expresiones booleanas. Son la implementación de la bifurcación en diagramas de flujo. En aplicaciones reales se usan para validar entradas, clasificar datos, aplicar descuentos según condiciones o determinar estados académicos.

9.5 Ciclos: for y while

Los ciclos permiten repetir bloques de instrucciones. El for se usa para un número conocido de iteraciones o para recorrer colecciones. El while se usa cuando la repetición depende de una condición variable. Ambos son fundamentales para procesar conjuntos de datos y evitar la duplicación de código.

9.6 Contadores y Acumuladores

Un contador es una variable que se incrementa en una cantidad fija para registrar cuántas veces ocurre un evento. Un acumulador es una variable que suma valores progresivamente durante un ciclo para calcular totales o promedios. La diferencia clave: el contador registra cuántas veces; el acumulador, cuánto en total. En LogicWeb UTA el módulo de Progreso usa ambos patrones internamente.

9.7 Funciones

Una función es un bloque de código con nombre que recibe parámetros, ejecuta un proceso y puede retornar un valor. Las funciones promueven modularidad, reutilización y mantenibilidad.



En Flask cada ruta es una función Python decorada con `@app.route`, demostrando la aplicación directa de este concepto.

9.8 Programación Orientada a Objetos (POO)

La POO organiza el software en clases (moldes) y objetos (instancias). Una clase define atributos (datos) y métodos (comportamientos). Sus pilares son abstracción, encapsulación, herencia y polimorfismo. En el contexto del proyecto, el módulo POO Básica ilustra la clase Ejercicio con su método `validar()`, directamente relacionada con la lógica de evaluación del sistema.

9.9 Python y Flask

Python es un lenguaje de programación interpretado, de alto nivel y propósito general, ampliamente utilizado en contextos educativos por su sintaxis clara, expresiva y cercana al lenguaje natural. Estas características facilitan el aprendizaje de la programación al reducir la complejidad sintáctica y permitir una mayor atención a la lógica del problema.

Flask, por su parte, es un microframework de desarrollo web basado en Python que permite construir aplicaciones ligeras, modulares y escalables. Aunque su filosofía es minimalista, ofrece los componentes esenciales para el desarrollo web, tales como el manejo de rutas mediante decoradores `@app.route`, la renderización de vistas HTML con **Jinja2**, la gestión de sesiones de usuario, el procesamiento de formularios y la integración con bases de datos como SQLite. Su flexibilidad y simplicidad lo convierten en una opción adecuada para proyectos académicos de mediana complejidad.

9.10 SQLite

SQLite es un sistema de gestión de bases de datos relacional embebido que no requiere un servidor independiente para su funcionamiento. Toda la información se almacena en un único archivo de base de datos, lo cual simplifica notablemente la instalación, el mantenimiento y la portabilidad del sistema. En proyectos educativos, SQLite resulta especialmente útil por su facilidad de integración con Python mediante el módulo `sqlite3`, su bajo consumo de recursos y su capacidad suficiente para soportar aplicaciones con un volumen moderado de usuarios y registros.

9.11 HTML5, CSS3 y JavaScript

CSS3 permite aplicar estilos visuales avanzados, tales como gradientes, sombras, diseño responsive, variables personalizadas y temas visuales modernos. En el caso del proyecto, CSS3



fue fundamental para implementar una interfaz en **modo oscuro**, con identidad visual coherente y adecuada para el contexto académico.

JavaScript complementa la capa de presentación al proporcionar interactividad del lado del cliente. Entre sus funciones principales se encuentran la validación básica de formularios, la manipulación del DOM, la respuesta inmediata a acciones del usuario y la mejora general de la experiencia de navegación.

9.12 PythonAnywhere

PythonAnywhere es una plataforma de alojamiento en la nube especializada en aplicaciones desarrolladas con Python. Proporciona soporte para **WSGI**, consolas Bash, gestión de archivos, despliegue de proyectos Flask y administración simplificada de entornos Python. Su facilidad de configuración y su disponibilidad de planes gratuitos la convierten en una herramienta ampliamente utilizada en entornos académicos y de prototipado.

9.13 Aplicativos Web Educativos

Los aplicativos web educativos son herramientas digitales orientadas a facilitar procesos de enseñanza-aprendizaje mediante la integración de contenidos, interacción y retroalimentación. Su principal ventaja radica en la accesibilidad desde distintos dispositivos sin necesidad de instalación local, así como en la posibilidad de brindar experiencias de aprendizaje autónomo, escalables y permanentemente disponibles.

En el ámbito de la programación, este tipo de aplicaciones permite combinar teoría, práctica y evaluación formativa en un solo entorno, lo que favorece la apropiación progresiva de conocimientos y la consolidación de habilidades de resolución de problemas.

9.14 Patrón MVC (Modelo-Vista-Controlador)

MVC es un patrón arquitectónico que divide una aplicación en tres componentes con responsabilidades claramente separadas:

- **Modelo:** gestiona los datos y la lógica de negocio. En LogicWeb UTA, corresponde a las funciones de acceso a SQLite (consultas, inserciones) y los cálculos del dominio (promedios, resultados de ejercicios, efectividad).
- **Vista:** es la capa de presentación que el usuario ve. Corresponde a las plantillas HTML (Jinja2) en la carpeta templates/: login.html, registro.html e index.html con todos sus bloques.
- **Controlador:** gestiona el flujo entre la Vista y el Modelo. En Flask, cada función decorada con `@app.route` actúa como controlador: recibe la petición HTTP, invoca el



Modelo para obtener o guardar datos, y decide qué Vista renderizar con qué información.

La adopción de MVC facilitó la división de trabajo entre los integrantes del equipo: un subgrupo trabajó en las plantillas HTML (Vista), otro en las rutas y lógica (Controlador) y otro en el esquema de datos (Modelo), permitiendo avances paralelos con mínima interferencia.

9.15 Metodología Iterativa Incremental

Además de MVC como patrón de diseño, el equipo adoptó una metodología de desarrollo iterativa e incremental como proceso de trabajo. En lugar de intentar construir todo el sistema de una vez, se definieron ciclos de desarrollo de aproximadamente una semana donde al final de cada ciclo existía un producto funcional y verificable:

- Iteración 1: autenticación (registro, login, logout) y estructura base.
- Iteración 2: módulos de información teórica (Variables → Funciones).
- Iteración 3: módulos POO, Comparar Lenguajes y actividades evaluadas por módulo.
- Iteración 4: Ejercicios Interactivos, Laboratorio Lógico y Mis Notas.
- Iteración 5: módulo de Progreso, panel de Reportes docente y despliegue en producción.

Este enfoque permitió detectar y corregir errores de forma temprana, ajustar prioridades según los avances y garantizar que el sistema en producción fuera estable al momento de la entrega.

10. Metodología de Desarrollo

Para la construcción del sistema **LogicWeb UTA** se aplicó una metodología de desarrollo **iterativa e incremental**, adecuada para proyectos académicos que requieren una evolución progresiva del producto mediante ciclos sucesivos de análisis, diseño, implementación, prueba y ajuste. Esta metodología permitió construir el sistema por módulos funcionales, validar resultados parciales y realizar mejoras continuas durante el proceso de desarrollo.

El proyecto combinó dos enfoques complementarios: MVC como patrón de diseño de software y el modelo iterativo-incremental como proceso de desarrollo del equipo.

10.1 Fase de Análisis

En esta fase se identificaron las necesidades académicas de los estudiantes de la asignatura **Algoritmos de Programación**, así como los requerimientos funcionales y no funcionales que debía satisfacer la aplicación. Se definieron los módulos principales del sistema, el stack tecnológico a utilizar (**Python, Flask, SQLite, HTML, CSS y JavaScript**) y el entorno de despliegue en **PythonAnywhere**. Asimismo, se estableció el propósito pedagógico de la



plataforma: reforzar el aprendizaje mediante contenido estructurado, ejercicios interactivos y seguimiento del desempeño.

10.2 Patrón MVC aplicado al proyecto

La estructura de archivos del proyecto respeta el patrón MVC de forma explícita:

```
PROYECTO_FINAL/
├── VISTA → templates/
│   ├── login.html      (pantalla de acceso)
│   ├── registro.html   (creación de cuenta)
│   └── index.html       (dashboard + todos los módulos Jinja2 blocks)
├── CONTROLADOR + MODELO → app.py
│   ├── Rutas @app.route (Controlador: recibe petición, decide
│   │                       respuesta)
│   ├── Funciones de BD   (Modelo: consultas y escrituras en SQLite)
│   └── Lógica de negocio (Modelo: cálculos, validaciones,
│   │                       retroalimentación)
└── BASE DE DATOS → base_datos.db
    ├── usuarios          (clase Usuarios)
    ├── practicas          (clase Practicas / historial)
    └── promedios          (clase Promedios)
└── actividades_modulo (clase Actividades por módulo)
```

10.3 División de trabajo del equipo

El equipo asignó responsabilidades alineadas con las capas del patrón MVC:

Integrante(s)	Capa MVC	Responsabilidad principal
Jahir Monar	Controlador + M	Arquitectura MVC, rutas Flask, lógica de negocio, acceso a BD y despliegue en PythonAnywhere.
Alexander Guaman	Vista	Plantillas HTML, diseño dark mode, navegación lateral y redacción del informe.
Alejandro Villacres	Contenidos + Documentación	Contenido teórico de módulos, revisión de actividades y documentación del informe.
Mateo Muyulema	Pruebas + Documentación	Pruebas funcionales del sistema y redacción de secciones del informe.

10.4 Fases del ciclo iterativo



1. Iteración 1 – Autenticación: login, registro, logout, sesiones Flask y estructura de BD inicial.
2. Iteración 2 – Módulos teóricos básicos: Variables, Condicionales, Bucles, Contadores, Acumuladores.
3. Iteración 3 – Módulos avanzados: Funciones, POO Básica, Comparar Lenguajes + 5 actividades por módulo.
4. Iteración 4 – Práctica: Ejercicios Interactivos, Laboratorio Lógico, Mis Notas.
5. Iteración 5 – Seguimiento: Módulo de Progreso, Panel Reportes docente, despliegue en producción.

10.5 Fase de Diseño

En la etapa de diseño se definió la arquitectura lógica del sistema, la estructura de navegación, el modelo de datos y la propuesta visual de la interfaz. Se adoptó una arquitectura de **tres capas**: presentación, lógica de negocio y datos. También se diseñó la base de datos relacional con tablas para usuarios, prácticas y promedios. En cuanto a la interfaz, se implementó un estilo visual de **tema oscuro** con acentos en color **teal (#00BCD4)**, sidebar lateral y distribución tipo dashboard, buscando una experiencia moderna, clara y consistente.

10.6 Fase de Implementación

La implementación se realizó de manera progresiva, iniciando con los módulos de autenticación y navegación principal, continuando con los contenidos teóricos y las actividades interactivas, y finalizando con el módulo de progreso y persistencia de datos. El desarrollo se llevó a cabo localmente en **Visual Studio Code**, utilizando un entorno virtual de Python para la gestión de dependencias. Durante esta fase se integraron las rutas Flask, las plantillas HTML con Jinja2, la lógica de validación y la comunicación con SQLite.

10.7 Fase de Pruebas

En esta fase se efectuaron pruebas funcionales sobre los procesos críticos del sistema, incluyendo el registro de usuarios, el inicio de sesión, la protección de rutas, la resolución de ejercicios, la persistencia de información y la visualización del progreso. Las pruebas se ejecutaron tanto en entorno local (localhost:5000) como en el entorno de producción desplegado en PythonAnywhere. El objetivo fue asegurar el correcto funcionamiento del sistema y detectar posibles errores de lógica, navegación o almacenamiento.

10.8 Fase de Despliegue



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Finalmente, el proyecto fue publicado en **PythonAnywhere**, configurando adecuadamente el archivo **WSGI**, el directorio fuente del proyecto y la inicialización de la base de datos en producción. Una vez completada la configuración, se verificó el acceso público al sistema desde la URL oficial y se ejecutaron pruebas finales de funcionamiento, navegación y persistencia, confirmando el despliegue exitoso de la aplicación



11. Requerimientos del Sistema

11.1 Requerimientos Funcionales Implementados

RF	Descripción	Estado
RF01	Registro de nuevos usuarios con usuario y contraseña.	✓ Implementado
RF02	Inicio de sesión con validación de credenciales.	✓ Implementado
RF03	Cierre de sesión con destrucción de la sesión activa.	✓ Implementado
RF04	Página principal (dashboard) con acceso a módulos.	✓ Implementado
RF05	Módulo de contenidos teóricos (6 temas).	✓ Implementado
RF06	Ejercicios interactivos con cálculo y retroalimentación.	✓ Implementado
RF07	Registro del historial de prácticas por usuario.	✓ Implementado
RF08	Visualización de progreso, efectividad y promedios.	✓ Implementado
RF09	Protección de rutas: solo usuarios autenticados acceden.	✓ Implementado
RF10	Registro y progreso de las actividades resueltas por el usuario	✓ Implementado
RF11	Visualización por parte del docente(o admin) de todo el progreso de todos los estudiantes hechas en un reporte	✓ Implementado

11.2 Requerimientos No Funcionales Implementados

- Interfaz **dark mode** con identidad visual coherente, tipografía legible y acentos cromáticos en teal para reforzar la experiencia de uso.
- Diseño **responsive**, adaptable a distintas resoluciones y dispositivos.



- Tiempo de respuesta adecuado para el entorno académico y de alojamiento en PythonAnywhere.
- Estructura de código modular y legible, con organización funcional de rutas y plantillas.
- Mensajes de validación y retroalimentación comprensibles para el estudiante.
- Uso de tecnologías gratuitas y open source, favoreciendo la sostenibilidad del proyecto.
- Accesibilidad web mediante navegador sin necesidad de instalación local.

12. Arquitectura del Sistema

12.1 Arquitectura de Tres Capas

El sistema sigue una arquitectura web de tres capas claramente separadas:

Capa	Tecnología utilizada	Responsabilidad
Presentación	HTML5, CSS3, JavaScript	Interfaz, navegación, formularios y estilo visual.
Lógica de negocio	Python 3 + Flask	Gestión de rutas, validaciones, sesiones, procesamiento de ejercicios, retroalimentación y control de acceso.
Datos	SQLite 3	Persistencia de usuarios, intentos, prácticas y promedios.
Hosting / despliegue	PythonAnywhere	Publicación de la aplicación, ejecución WSGI y disponibilidad pública del sistema.

La arquitectura implementada favorece la modularidad del proyecto, ya que cada capa puede ser analizada, mejorada o ampliada con relativa independencia. Además, permite una trazabilidad clara entre las acciones del usuario en la interfaz, la lógica de procesamiento en Flask y la persistencia de la información en SQLite.

12.2 Estructura de Archivos del Proyecto

La organización del proyecto en el sistema de archivos es la siguiente:

```
PROYECTO_FINAL/  
|  
└── templates/      # Plantillas HTML renderizadas con Jinja2
```



```
| |— index.html      # Dashboard principal posterior al login
| |— login.html     # Página de inicio de sesión
| |— registro.html  # Página de registro
| |— ...           # Vistas adicionales de módulos y progreso
|
|— static/         # Archivos CSS, JS e imágenes del sistema
| |— css/
| |— js/
| |— img/
|
|— app.py          # Aplicación principal Flask: rutas, lógica y conexión BD
|— base_datos.db   # Base de datos SQLite
|— venv/           # Entorno virtual local de desarrollo
```

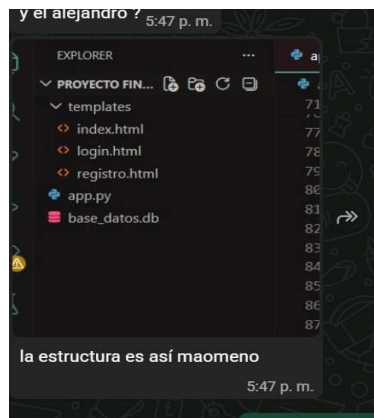


Figura 1. Estructura del proyecto en Visual Studio Code

12.3 Flujo de Navegación

El flujo general de navegación del sistema es:

1. El usuario accede a <https://logicweb.pythonanywhere.com> y visualiza la pantalla de login.
2. Si no tiene cuenta, navega a la página de registro y crea su usuario.
3. Al iniciar sesión correctamente, Flask crea una sesión y redirige al dashboard (index.html).
4. Desde el dashboard el usuario accede a cualquier módulo de contenido o ejercicio.
5. Cada ejercicio resuelto se guarda en la tabla 'practicass' de SQLite con el resultado y retroalimentación.
6. El módulo de Progreso consulta la base de datos y muestra estadísticas actualizadas.



7. Al cerrar sesión, Flask elimina la sesión activa y redirige al login.

12.4 Mapa de rutas Flask (Capa Controlador)

Ruta	Método(s)	Controlador: función y descripción
/	GET	Redirige a /login o /index según sesión activa.
/login	GET,POST	Muestra formulario. POST: valida en BD, crea session['usuario'].
/registro	GET,POST	Muestra formulario. POST: inserta usuario nuevo en BD.
/logout	GET	session.clear() → redirige a login.
/index	GET	Dashboard. Requiere sesión. Renderiza tarjetas de módulos.
/progreso	GET	Consulta practicas y promedios del usuario en sesión.
/variables	GET,POST	Teoría + 5 actividades. POST evalúa y guarda en actividades_modulo.
/condicionales	GET,POST	Teoría + 5 actividades con escenarios de decisión lógica.
/bucles	GET,POST	Teoría + 5 actividades sobre for, while y recorridos.
/contadores	GET,POST	Teoría + 5 actividades de conteo de eventos.
/acumuladores	GET,POST	Teoría + 5 actividades de suma progresiva.
/funciones	GET,POST	Teoría + 5 actividades de def, parámetros y return.
/poo	GET,POST	Teoría + conceptos clave. Clase Ejercicio con validar().
/comparar	GET	Condicional en Python, C++ y Java.
/ejercicios	GET,POST	5 ejercicios de vida real. POST: calcula y guarda en practicas.
/laboratorio	GET,POST	Tablas de verdad + constructor EPS. Guarda en practicas.
/mis_notas	GET	Bloc de apuntes (sessionStorage JS).
/reportes	GET	Panel docente. Solo usuario 'profe_ruben' puede acceder.

13. Diseño Funcional del Sistema

13.1 Módulo de Autenticación

El módulo de autenticación gestiona el ciclo completo de la sesión del usuario:

- Registro: el formulario en registro.html envía usuario y contraseña al endpoint /registro (POST). Flask valida que el usuario no exista, y si es nuevo, lo inserta en la tabla usuarios de SQLite.



- Login: el formulario en login.html envía las credenciales al endpoint /login (POST). Flask consulta la tabla usuarios, compara la contraseña y, si es correcta, almacena el usuario en la sesión de Flask (session['usuario']).
- Logout: el endpoint /logout elimina la clave de sesión y redirige al login.
- Protección de rutas: un decorador personalizado verifica que session['usuario'] exista antes de permitir el acceso al dashboard y módulos. Si no hay sesión activa, redirige al login.

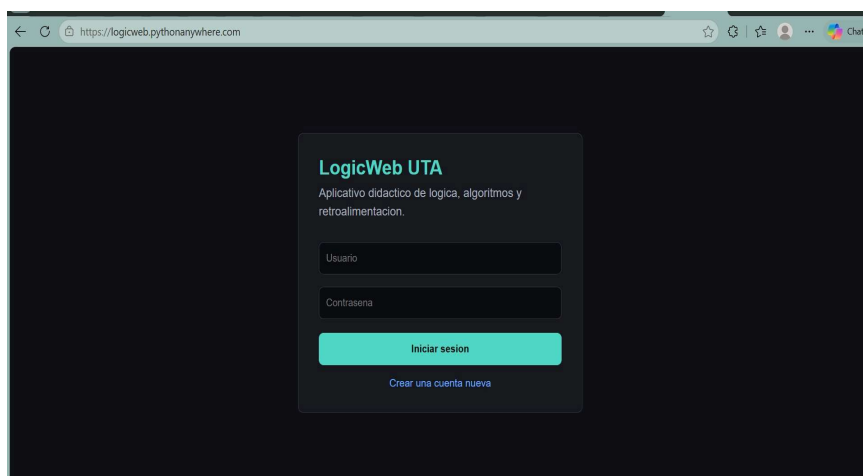


Figura 2. Pantalla de inicio de sesión – LogicWeb UTA

13.2 Módulo de Inicio (Dashboard)

El dashboard principal se presenta una vez que el usuario ha iniciado sesión correctamente. Este módulo funciona como centro de navegación y síntesis general del sistema, mostrando la identidad del usuario activo, una descripción breve del propósito de la plataforma y accesos directos a los diferentes apartados disponibles.

Desde el punto de vista funcional, el dashboard mejora la usabilidad del sistema al centralizar la navegación, permitiendo un acceso ágil tanto a contenidos teóricos como a ejercicios y módulos de seguimiento. Su diseño responde a una lógica de panel principal de trabajo, donde el estudiante puede identificar rápidamente las opciones disponibles.

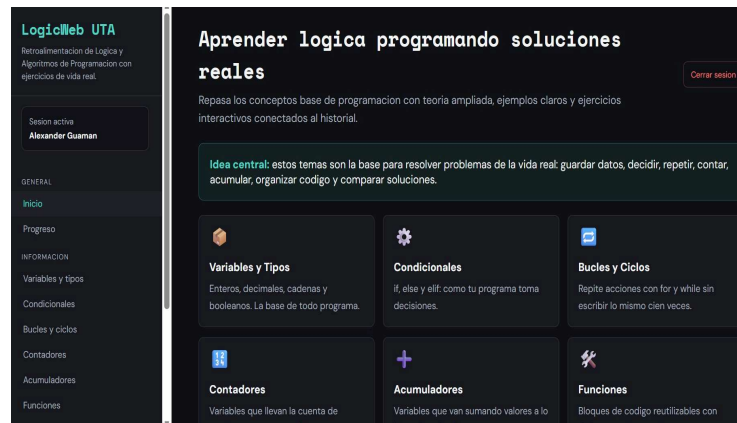


Figura 3. Dashboard principal – acceso a módulos de aprendizaje

Análisis funcional: El dashboard centraliza la experiencia de navegación y reduce la fricción en el uso del sistema, facilitando el paso entre teoría, práctica y seguimiento del rendimiento.

Contribución al proyecto: Mejora la experiencia de usuario y refuerza el objetivo de ofrecer un entorno educativo organizado, accesible y funcional.

13.3 Módulos de Contenido Teórico

Se implementaron seis módulos de contenido, accesibles desde el sidebar y el dashboard:

Módulo	Contenido principal
Variables y Tipos	Enteros, decimales, cadenas y booleanos. Declaración y uso de variables en Python.
Condicionales	Estructuras if, else y elif. Toma de decisiones en programas.
Bucles y Ciclos	Ciclos for y while. Repetición de acciones e iteración sobre colecciones.
Contadores	Variables que registran la frecuencia de eventos o iteraciones.
Acumuladores	Variables que suman o acumulan valores de forma progresiva.
Funciones	Bloques de código reutilizables con parámetros y valor de retorno.
POO Básica	Introducción a clases, objetos, atributos y métodos en Python.

Cada módulo incluye una definición conceptual, una explicación ampliada, ejemplos en código y relaciones con situaciones prácticas o con el propio funcionamiento del sistema.



Figura 4. Módulo “Bucles y Ciclos”

Descripción: La interfaz expone el concepto de estructuras repetitivas mediante secciones diferenciadas para *for* y *while*, acompañadas de ejemplos en Python y una explicación de su importancia.

Análisis funcional: El módulo permite al estudiante comparar dos tipos fundamentales de repetición y comprender sus usos en problemas computacionales concretos.

Contribución al proyecto: Refuerza la comprensión de estructuras iterativas, esenciales para el desarrollo del pensamiento algorítmico.

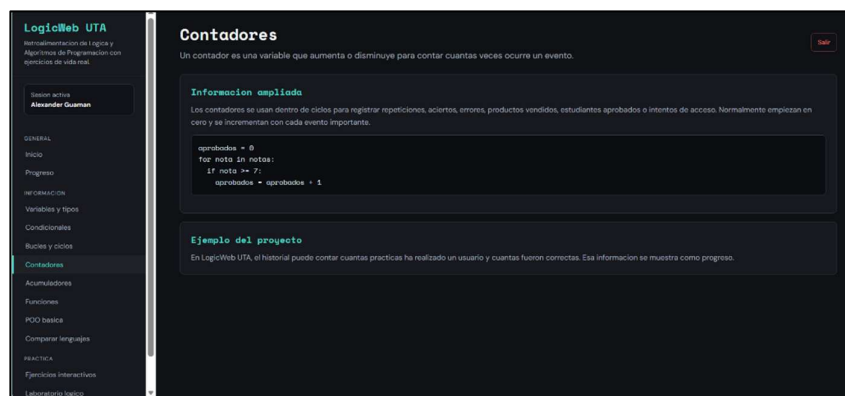


Figura 5. Módulo “Contadores”

Descripción: La pantalla explica el uso de contadores como variables que registran la ocurrencia de eventos, acompañada de ejemplos prácticos y relación con el historial de prácticas del sistema.

Análisis funcional: Facilita la comprensión de cómo cuantificar eventos dentro de procesos iterativos y cómo dicho concepto se aplica dentro del mismo aplicativo.

Contribución al proyecto: Vincula teoría y práctica, favoreciendo una comprensión aplicada del seguimiento de resultados.

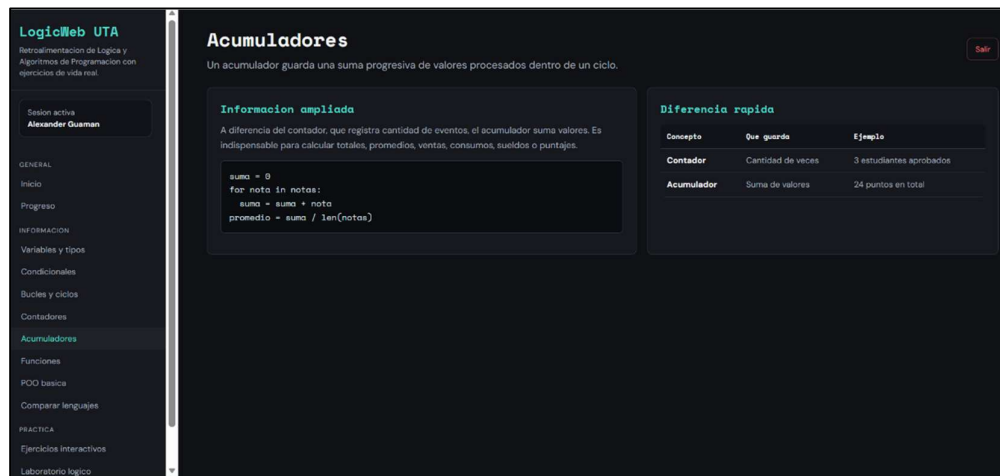


Figura 6. Módulo “Acumuladores”

Descripción: La interfaz presenta el concepto de acumulador, su diferencia frente al contador y un ejemplo de cálculo de promedios mediante sumas progresivas.

Análisis funcional: El módulo ayuda al estudiante a diferenciar conceptos que suelen confundirse en etapas iniciales del aprendizaje.

Contribución al proyecto: Mejora la claridad conceptual y fortalece la base lógica necesaria para resolver ejercicios numéricos.

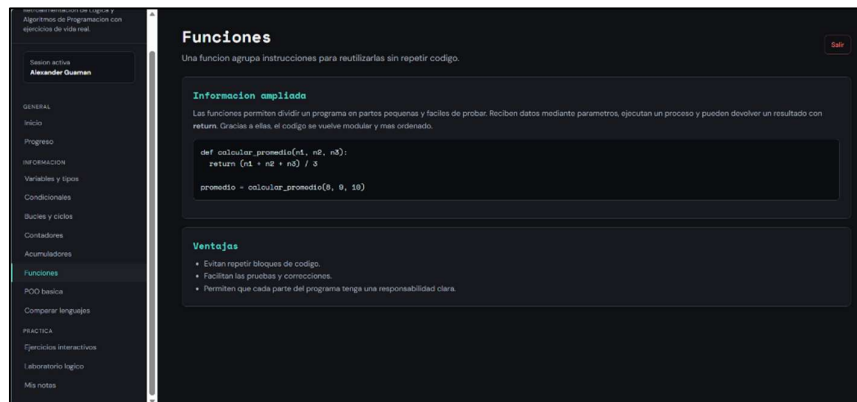


Figura 7. Módulo “Funciones”

Descripción: La pantalla desarrolla el concepto de funciones como bloques reutilizables de código, mostrando sintaxis, parámetros, retorno y ventajas de modularidad.

Análisis funcional: Promueve la comprensión de la descomposición funcional de problemas y de la reutilización de lógica dentro del desarrollo de software.

Contribución al proyecto: Refuerza buenas prácticas de programación estructurada y modular.

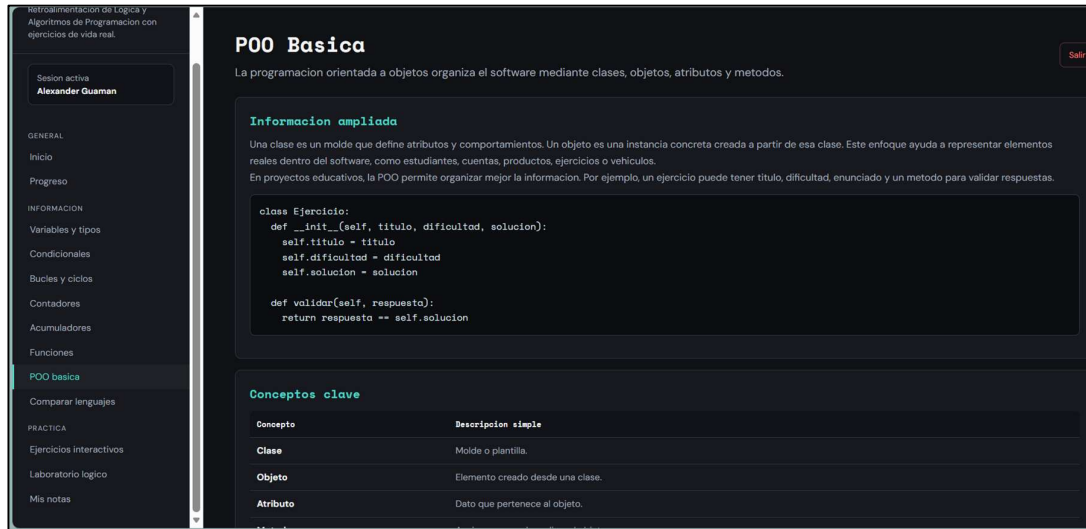


Figura 8. Módulo “POO Básica”

***Descripción:** La figura muestra el contenido formativo sobre programación orientada a objetos, incluyendo definiciones, ejemplo de clase en Python y tabla de conceptos clave.*

Análisis funcional: Introduce al estudiante en un paradigma de programación complementario, relacionando la teoría con ejemplos directamente comprensibles.

Contribución al proyecto: Amplía el alcance formativo de la plataforma y fortalece la continuidad del aprendizaje más allá de la programación estructurada básica.

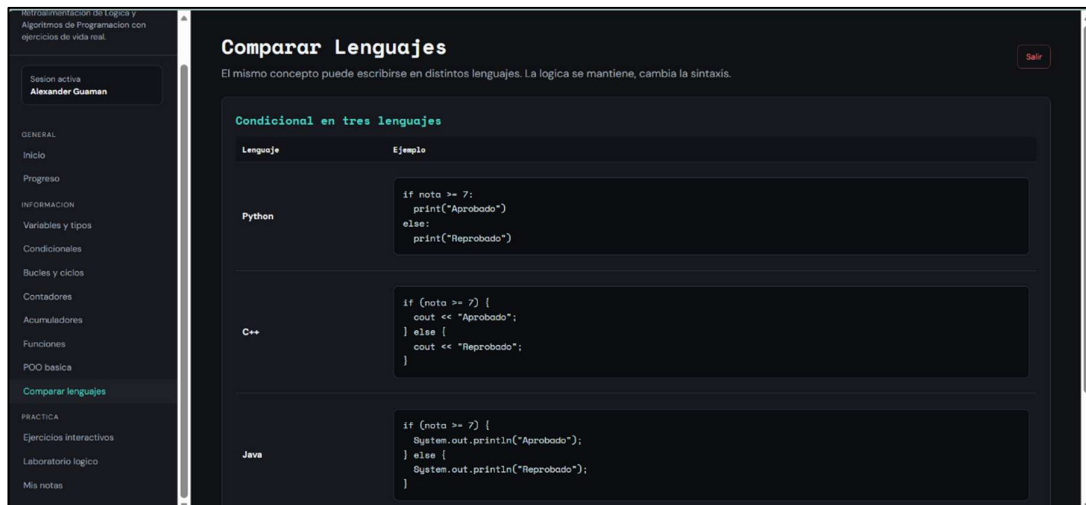


Figura 9. Módulo “Comparar Lenguajes”

***Descripción:** La pantalla compara una misma estructura lógica condicional en Python, C++ y Java, resaltando que la lógica permanece mientras la sintaxis cambia.*



Análisis funcional: Facilita la transferencia de conocimientos entre lenguajes y ayuda al estudiante a distinguir entre razonamiento algorítmico y forma sintáctica.

Contribución al proyecto: Fortalece la comprensión abstracta de la lógica computacional y su independencia relativa del lenguaje utilizado.

13.4 Módulo de Ejercicios Interactivos

Los ejercicios interactivos constituyen uno de los componentes centrales de **LogicWeb UTA**, ya que permiten al estudiante aplicar los conocimientos adquiridos en contextos concretos. Cada actividad presenta un enunciado, campos de entrada, botones de ejecución y una respuesta procesada por el sistema con retroalimentación inmediata.

Los ejercicios implementados incluyen:

- Cálculo de promedio de notas de un estudiante.
- Cálculo de sueldo con horas extras.
- Determinación de descuentos en una tienda.
- Clasificación de edades y categorías.
- Control de ventas y cálculo de totales.
- Inventario básico de productos.
- Consumo de energía eléctrica en el hogar.

Desde el punto de vista técnico, estos ejercicios siguen un flujo **entrada** → **procesamiento** → **salida**, reforzando explícitamente el enfoque algorítmico de la asignatura. Además, cada intento puede almacenarse en la base de datos junto con el módulo, el ejercicio, el resultado, la retroalimentación y la fecha de ejecución.



The screenshot displays the 'Ejercicios interactivos' module on the LogicWeb UTA platform. The interface is dark-themed and includes a sidebar with navigation options like 'Inicio', 'Progreso', 'Información', and 'Práctica'. The main area features several exercise cards: 'Promedio académico' with input fields for three grades and a 'Calcular y guardar' button; 'Clasificador de edad' with an age input and 'Clasificar'/'Guardar practica' buttons; 'Sueldo con horas extra' with inputs for normal hours, hourly rate, and extra hours, plus 'Resolver'/'Guardar practica' buttons; 'Consumo de agua o energía' with inputs for previous/actual readings and unit value, plus 'Resolver'/'Guardar practica' buttons; and 'Descuento en tienda' with inputs for subtotal, client type, and coupon. A 'Mini quiz' section is also visible at the bottom.

Figura 10. Módulo de Ejercicios Interactivos

Descripción: La figura muestra varias tarjetas de ejercicios basados en situaciones reales, cada una con formularios de entrada, botones de resolución y opciones de guardado.

Análisis funcional: Esta interfaz permite al estudiante resolver problemas de forma práctica, recibir retroalimentación inmediata y registrar sus intentos en la base de datos.

Contribución al proyecto: Constituye la evidencia principal del enfoque aplicado del sistema, al transformar la teoría en experiencia interactiva y medible.

13.5 Módulo de Progreso

El módulo de progreso consulta la base de datos y presenta al usuario tres métricas principales:

- Prácticas registradas: total de ejercicios intentados.
- Respuestas correctas: cantidad de ejercicios resueltos correctamente.
- Efectividad aproximada: porcentaje de aciertos sobre el total de intentos.

Adicionalmente muestra el Historial de prácticas (tabla con módulo, ejercicio, resultado, retroalimentación y fecha) y la tabla de Promedios calculados.

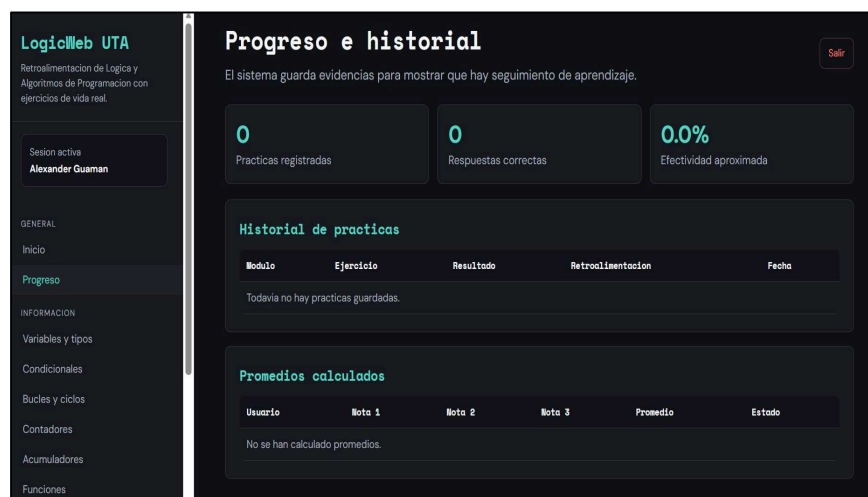


Figura 11. Módulo de Progreso e Historial de prácticas

Descripción: La figura presenta el panel de seguimiento del usuario, incluyendo métricas acumuladas, historial de prácticas y resultados almacenados.

Análisis funcional: El módulo transforma los registros almacenados en información útil para monitorear el aprendizaje y evidenciar el avance del estudiante.

Contribución al proyecto: Refuerza la dimensión pedagógica del sistema, al incorporar evaluación formativa y seguimiento objetivo del desempeño.

13.6 Módulo de Laboratorio Lógico

El módulo **Laboratorio Lógico** incorpora herramientas específicas para el razonamiento formal y algorítmico, como el generador de tablas de verdad y el constructor EPS (Entrada, Proceso y Salida). Este apartado amplía el alcance del sistema más allá de los ejercicios convencionales, incorporando recursos de apoyo conceptual y de preparación para exposiciones.

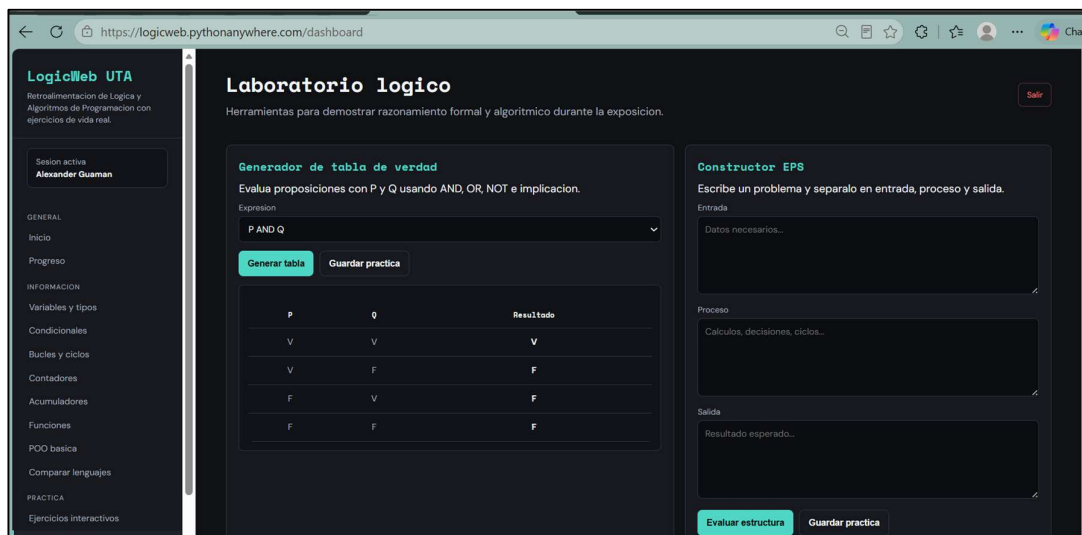


Figura 12. Laboratorio lógico con tabla de verdad y constructor EPS

Descripción: La interfaz presenta herramientas para evaluar expresiones lógicas con operadores booleanos y para estructurar problemas mediante el esquema Entrada-Proceso-Salida.

Análisis funcional: Este módulo fortalece la comprensión del razonamiento lógico formal y la estructuración previa de algoritmos, dos elementos clave en el aprendizaje de programación.

Contribución al proyecto: Aporta valor pedagógico adicional al sistema, integrando recursos que favorecen tanto la explicación como la práctica estructurada.

13.7 Módulo de Notas de Apoyo

El apartado **Mis notas** actúa como un espacio complementario para que el usuario registre ideas, observaciones o apuntes útiles para la preparación de la defensa del proyecto o del estudio personal.

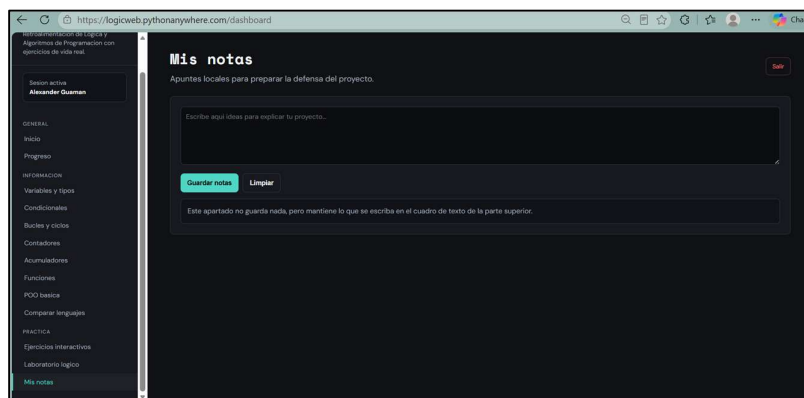


Figura 13. Módulo “Mis notas”

Descripción: La figura muestra un área de texto destinada al registro de ideas y apuntes locales relacionados con la exposición y comprensión del proyecto.

Análisis funcional: Aunque no representa una funcionalidad central de persistencia académica, este módulo reduce la necesidad de herramientas externas y fortalece la experiencia integral del usuario dentro de la plataforma.

Contribución al proyecto: Apoya la preparación de la defensa y fomenta la organización autónoma del aprendizaje.

13.8 Panel de Reportes Docente

El panel de Reportes Docente es la funcionalidad más avanzada del sistema en términos de gestión académica. Accesible únicamente con las credenciales del docente (usuario: profe_ruben, contraseña: 1234), presenta una vista consolidada del desempeño de todos los estudiantes registrados en tiempo real, permitiendo al docente tomar decisiones pedagógicas basadas en datos concretos y verificables del sistema en producción.



13.8.1 Acceso y autenticación del panel docente

El acceso al panel de reportes está protegido por un mecanismo de doble verificación implementado directamente en la ruta Flask /reportes. El controlador primero valida que exista una sesión activa mediante `session.get('usuario')`; si no hay sesión, redirige al login. En segundo lugar, verifica que el usuario autenticado sea específicamente 'profe_ruben', único usuario con privilegios de docente en el sistema actual. Si un estudiante intenta acceder directamente a la URL /reportes estando autenticado, el sistema lo redirige al dashboard con un mensaje de acceso denegado. Esta arquitectura de control de acceso por roles, aunque simple, implementa el principio de mínimo privilegio: cada usuario solo accede a las funcionalidades que le corresponden según su rol.

13.8.2 Estructura del reporte: Sección de Resumen por Estudiante y Módulo

El reporte se estructura en dos secciones claramente diferenciadas. La primera es el Resumen por estudiante y módulo: una tabla agregada que muestra, para cada combinación de usuario y módulo, el total de actividades completadas, las respuestas correctas, las incorrectas y la calificación porcentual calculada automáticamente ($\text{correctas} / \text{total} \times 100$). Esta sección es generada mediante una consulta SQL con JOIN entre las tablas `actividades_modulo` y `usuarios`, agrupando por usuario y módulo con `GROUP BY`. La calificación se calcula con `ROUND(SUM(CASE WHEN resultado='Correcto' THEN 1.0 ELSE 0 END) / COUNT(*) * 100, 1)` garantizando un resultado preciso con un decimal. Este resumen permite al docente identificar rápidamente en qué módulos el grupo completo presenta mayor dificultad y qué estudiantes requieren atención individualizada.

13.8.3 Estructura del reporte: Sección de Detalle de Respuestas

La segunda sección es el Detalle completo de respuestas: una tabla con registro individual de cada interacción del estudiante con el sistema. Cada fila contiene la fecha y hora exacta del intento (campo `fecha_hora` con `DEFAULT CURRENT_TIMESTAMP`), el nombre del usuario, el módulo al que pertenece la actividad, el número o nombre de la actividad específica, la respuesta ingresada por el estudiante, el resultado obtenido (Correcto / Incorrecto) y la retroalimentación personalizada generada por el sistema. Esta vista detallada permite al docente analizar no solo el resultado final, sino el proceso de respuesta de cada estudiante: si un alumno falló repetidamente la misma actividad, si ingresó valores incorrectos de forma sistemática o si su desempeño mejoró progresivamente a través de los intentos. Esta información cualitativa y cuantitativa tiene un alto valor pedagógico para la toma de decisiones docentes.

13.8.4 Tabla `actividades_modulo`: estructura y persistencia de datos

El módulo de reportes se alimenta exclusivamente de la tabla `actividades_modulo`, que almacena cada interacción de un estudiante con las actividades evaluadas de los módulos teóricos. Su



estructura SQL es: id (INTEGER PRIMARY KEY AUTOINCREMENT), usuario (TEXT NOT NULL, FK → usuarios.usuario), modulo (TEXT NOT NULL), actividad_num (INTEGER NOT NULL representando la actividad 1 a 5 dentro del módulo), respuesta (TEXT NOT NULL con la respuesta literal del estudiante), resultado (TEXT NOT NULL con valor 'Correcto' o 'Incorrecto'), retroalimentacion (TEXT con el mensaje explicativo generado por el sistema) y fecha (DATETIME DEFAULT CURRENT_TIMESTAMP). Cada vez que un estudiante responde una actividad dentro de cualquier módulo teórico (Variables, Condicionales, Bucles, Contadores, Acumuladores o Funciones), Flask ejecuta un INSERT INTO actividades_modulo registrando todos estos campos. La ausencia de restricción UNIQUE sobre (usuario, modulo, actividad_num) es intencional: permite múltiples intentos por actividad, reflejando de manera fiel el proceso de aprendizaje iterativo del estudiante.

13.8.5 Implementación técnica: consultas SQL y lógica Flask

La ruta /reportes ejecuta dos consultas SQL independientes contra la base de datos SQLite. La primera consulta genera el resumen agrupado: SELECT usuario, modulo, COUNT(*) as total, SUM(CASE WHEN resultado='Correcto' THEN 1 ELSE 0 END) as correctas, SUM(CASE WHEN resultado='Incorrecto' THEN 1 ELSE 0 END) as incorrectas, ROUND(SUM(CASE WHEN resultado='Correcto' THEN 1.0 ELSE 0 END)/COUNT(*)*100, 1) as calificacion FROM actividades_modulo GROUP BY usuario, modulo ORDER BY usuario, modulo. La segunda consulta recupera el detalle completo: SELECT * FROM actividades_modulo ORDER BY fecha DESC, mostrando primero los intentos más recientes. Ambos conjuntos de resultados se pasan a la plantilla Jinja2 como variables del contexto (resumen y detalle), donde son renderizados en tablas HTML con formato visual consistente con el tema dark mode del sistema. La plantilla utiliza bucles {% for row in resumen %} y {% for row in detalle %} para iterar sobre los registros.

13.8.6 Funcionalidad de impresión y exportación del reporte

El sistema incluye un botón de impresión implementado con window.print() que genera una vista limpia del reporte para ser guardada en PDF o impresa físicamente, facilitando la entrega de evidencias académicas al departamento o al portafolio docente. La funcionalidad de impresión está optimizada mediante reglas CSS @media print que ocultan el sidebar de navegación, los botones de acción, el encabezado visual y todos los elementos decorativos del tema dark mode, presentando únicamente las tablas de datos sobre fondo blanco con tipografía negra estándar. Esto garantiza que el documento impreso o PDF resultante sea formalmente legible y adecuado para entornos institucionales. Como méjora futura, se propone integrar la generación de reportes en formato Excel mediante la librería openpyxl de Python, lo que permitiría al docente descargar

directamente un archivo .xlsx con todos los datos del reporte para análisis externos, cálculo de calificaciones parciales o entrega a coordinación académica.

13.8.7 Valor pedagógico e impacto del módulo de Reportes

El módulo de Reportes Docente convierte a LogicWeb UTA en una herramienta de gestión del aprendizaje (LMS simplificado), y no solo en una plataforma de práctica individual. Desde una perspectiva pedagógica, los datos del reporte permiten al docente: (1) Identificar qué conceptos generan mayor cantidad de errores en el grupo, orientando la planificación de clases de refuerzo focalizadas. (2) Detectar estudiantes con bajo desempeño en módulos específicos para brindar atención personalizada o asignar actividades de nivelación. (3) Reconocer patrones de error sistemático, como respuestas incorrectas recurrentes en la misma actividad, lo que puede indicar una explicación insuficiente de un concepto específico. (4) Verificar la participación activa de todos los estudiantes en la plataforma, funcionando como un registro de asistencia y cumplimiento de actividades. (5) Construir rúbricas de evaluación más precisas, basadas en el desempeño real medido por el sistema. Este componente, desarrollado en el Sprint 5 del proceso iterativo del equipo, representa la integración más compleja del proyecto, ya que combina consultas SQL avanzadas, lógica de acceso por roles, renderizado de datos dinámicos con Jinja2 y funcionalidades de exportación, demostrando un nivel de madurez técnica significativo para un proyecto formativo de primer semestre.



Figura 16. Panel Reportes Docente – desempeño de todos los participantes con calificaciones e historial detallado

Datos verificados del sistema en producción: el docente profe_ruben puede ingresar a <https://logicweb.pythonanywhere.com> con la contraseña 1234 y visualizar en tiempo real el reporte de todos los estudiantes que han completado actividades, con su módulo, actividad, respuesta, resultado y retroalimentación correspondiente.



14. Diseño de Datos

14.1 Modelo Entidad-Relación

La base de datos base_datos.db implementa la capa Modelo del patrón MVC con cuatro tablas que representan las entidades principales del sistema. Todas se inicializan en la función init_db() de app.py al arrancar la aplicación.

Clase usuarios – gestión de autenticación

Campo	Tipo	Descripción	Restricción
Id	INTEGER	Identificador único del usuario	PK, AUTOINCREMENT
Usuario	TEXT	Nombre de usuario para login	NOT NULL, UNIQUE
contrasena	TEXT	Contraseña del usuario	NOT NULL

Clase practicas – historial de ejercicios interactivos y laboratorio

Campo	Tipo	Descripción	Restricción
Id	INTEGER	Identificador único del intent	PK, AUTOINCREMENT
Usuario	TEXT	Usuario que realizó la práctica	FK → usuarios.usuario
Modulo	TEXT	Nombre del módulo practicado	NOT NULL
ejercicio	TEXT	Descripción del ejercicio	NOT NULL
resultado	TEXT	Correcto / Incorrecto	NOT NULL
retroalimentacion	TEXT	Mensaje explicativo al estudiante	
Fecha	DATETIME	Fecha y hora del intento	DEFAULT NOW

Clase promedios – cálculos académicos

Campo	Tipo	Descripción	Restricción
-------	------	-------------	-------------



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Id	INTEGER	Identificador único del registro	PK, AUTOINCREMENT
Usuario	TEXT	Usuario dueño del registro	FK → usuarios.usuario
nota1	REAL	Primera nota del estudiante	NOT NULL
nota2	REAL	Segunda nota del estudiante	NOT NULL
nota3	REAL	Tercera nota del estudiante	NOT NULL
promedio	REAL	Promedio calculado	NOT NULL
Estado	TEXT	Aprobado / Reprobado	NOT NULL

El diseño de datos responde a una estructura simple pero funcional, adecuada para un entorno académico. La separación entre usuarios, prácticas y promedios permite mantener integridad lógica, organizar los registros por tipo de información y facilitar la consulta del historial y las métricas de progreso.

Clase actividades_modulo – 5 actividades por módulo teórico

Campo	Tipo	Descripción	Restricción
Id	INTEGER	Identificador único	PK, AUTOINCREMENT
Usuario	TEXT	FK: usuario que respondió	FK → usuarios.usuario
Modulo	TEXT	Nombre del módulo	NOT NULL
actividad_num	INTEGER	Número de actividad (1-5)	NOT NULL
respuesta	TEXT	Respuesta del estudiante	NOT NULL
resultado	TEXT	Correcto / Incorrecto	NOT NULL
retroalimentacion	TEXT	Explicación del resultado	
Fecha	DATETIME	Fecha del intento	DEFAULT CURRENT_TIMESTAMP



```
Bash console 46996161
jl:05 ~ % ls
README.txt  __pycache__  app.py  base_datos.db  flask_app.py  templates
jl:06 ~ % sqlite3 base_datos.db
sqlite version 3.37.2 2022-01-06 13:25:41
Enter ".help" for usage hints.
sqlite> .tables
intentos  practicas  usuarios
sqlite> SELECT * FROM usuarios
...>
Error: in prepare, near "SELECT": syntax error (1)
sqlite> SELECT * FROM usuarios;
1|profe_ruben|1234|docente
3|shairi|12345|estudiante
1|Alex|12345|estudiante
3|zink|20161990|estudiante
3|Alexander guaman|1850474642|estudiante
sqlite>
```

Figura 14. Estructura de la base de datos SQLite

Descripción: La figura representa las tablas principales del sistema, sus campos y la relación entre usuarios, prácticas y promedios.

Análisis funcional: El diseño evidencia una estructura relacional suficiente para soportar autenticación, registro de prácticas y seguimiento académico del usuario.

Contribución al proyecto: Garantiza la persistencia de datos y la trazabilidad del desempeño de cada estudiante dentro del sistema.

15. Proceso de Implementación Paso a Paso

Esta sección documenta de manera detallada cada etapa del proceso de desarrollo e implementación del sistema LogicWeb UTA, desde la configuración del entorno hasta el despliegue en producción.

PASO 1 – Configuración del Entorno de Desarrollo

Se instaló Python 3 y Visual Studio Code como entorno de desarrollo. Se creó un entorno virtual Python para aislar las dependencias del proyecto:

```
python -m venv venv
venv\Scripts\activate      # Windows
pip install flask
```

Se verificó la instalación de Flask ejecutando `flask --version` y se confirmó que el entorno funcionaba correctamente.

PASO 2 – Creación de la Estructura del Proyecto

Se organizó la carpeta del proyecto con la estructura mínima requerida por Flask:

```
mkdir PROYECTO_FINAL
cd PROYECTO_FINAL
```



```
mkdir templates  
touch app.py
```

Esta estructura separa claramente el backend (app.py) de las plantillas HTML (carpeta templates/), siguiendo las convenciones de Flask para el motor de plantillas Jinja2.

PASO 3 – Diseño e Implementación de la Base de Datos(Cñases)

Se diseñó el esquema de base de datos SQLite con tres tablas: usuarios, practicas y promedios. La inicialización de la base de datos se implementó directamente en app.py mediante la función `init_db()`:

```
import sqlite3  
  
def init_db():  
    conn = sqlite3.connect('base_datos.db')  
    c = conn.cursor()  
    # Tabla de usuarios  
    c.execute("""CREATE TABLE IF NOT EXISTS usuarios (  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        usuario TEXT UNIQUE NOT NULL,  
        contrasena TEXT NOT NULL)""")  
    # Tabla de prácticas  
    c.execute("""CREATE TABLE IF NOT EXISTS practicas (  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        usuario TEXT NOT NULL,  
        modulo TEXT NOT NULL,  
        ejercicio TEXT NOT NULL,  
        resultado TEXT NOT NULL,  
        retroalimentacion TEXT,  
        fecha DATETIME DEFAULT CURRENT_TIMESTAMP)""")  
    # Tabla de promedios calculados  
    c.execute("""CREATE TABLE IF NOT EXISTS promedios (  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        usuario TEXT NOT NULL,  
        nota1 REAL, nota2 REAL, nota3 REAL,  
        promedio REAL, estado TEXT)""")  
    conn.commit()
```



```
conn.close()
```

Esta función se llama una vez al iniciar la aplicación mediante `with app.app_context(): init_db()`, garantizando que las tablas existan antes de cualquier operación.

PASO 4 – Implementación del Backend Flask (app.py)

Se desarrolló `app.py` con todas las rutas del sistema. Las rutas principales implementadas son:

Ruta	Método	Descripción
/	GET	Redirige al login si no hay sesión, al dashboard si hay sesión activa.
/login	GET/POST	Muestra el formulario de login y procesa las credenciales.
/registro	GET/POST	Muestra el formulario de registro y crea nuevos usuarios.
/logout	GET	Destruye la sesión activa y redirige al login.
/index	GET	Dashboard principal (requiere sesión activa).
/progreso	GET	Módulo de progreso: consulta practicas y promedios del usuario.
/variables	GET/POST	Módulo de Variables y Tipos con ejercicio interactivo.
/condicionales	GET/POST	Módulo de Condicionales con ejercicio de descuento.
/bucles	GET/POST	Módulo de Bucles con ejercicio de suma acumulada.
/contadores	GET/POST	Módulo de Contadores con ejercicio de ventas.
/acumuladores	GET/POST	Módulo de Acumuladores con ejercicio de nómina.
/funciones	GET/POST	Módulo de Funciones con ejercicio de IMC.

Cada ruta de módulo verifica la sesión activa, procesa el formulario POST (si existe), calcula el resultado, genera la retroalimentación, guarda el intento en la tabla practicas y renderiza la plantilla con los resultados.

PASO 5 – Implementación del Frontend

Se crearon tres plantillas HTML (`login.html`, `registro.html`, `index.html`) utilizando Jinja2 para la herencia de plantillas. El diseño visual implementado sigue un tema dark mode con las siguientes características:

- Fondo principal: #1a1a2e (azul muy oscuro)



- Color de acento primario: #00BCD4 (teal/cian)
- Tipografía monoespaciada para elementos de código y datos
- Sidebar de navegación fijo a la izquierda con los módulos del sistema
- Área de contenido principal con tarjetas y tablas
- Botón de cierre de sesión en la esquina superior derecha

Los formularios de ejercicios utilizan JavaScript del lado del cliente para validación básica de campos y mejora de la experiencia de usuario.

PASO 6 – Cinco actividades por módulo

Para cada módulo se definieron 5 actividades con pregunta, respuesta esperada y retroalimentación específica. El controlador evalúa la respuesta del estudiante, genera el mensaje de retroalimentación y guarda en `actividades_modulo`.

PASO 7 – Panel de Reportes Docente

Se implementó la ruta `/reportes` con acceso restringido al usuario 'profe_ruben'. La consulta JOIN entre `actividades_modulo` y `usuarios` genera el resumen por estudiante y módulo, calculando correctas, incorrectas y calificación porcentual.

PASO 8 – Pruebas en Entorno Local

Se ejecutó el servidor de desarrollo Flask con:

```
python app.py
```

El sistema quedó disponible en `http://localhost:5000`. Se realizaron pruebas de:

- Registro de nuevos usuarios y validación de usuario duplicado.
- Inicio y cierre de sesión correcto.
- Acceso a todos los módulos de contenido.
- Resolución de ejercicios y verificación del guardado en SQLite.
- Visualización del historial y progreso en el módulo de Progreso.
- Protección de rutas: intento de acceso sin sesión → redirección a login.

PASO 8 – Preparación para Despliegue en PythonAnywhere

Para el despliegue en PythonAnywhere se realizaron los siguientes pasos de preparación:

8. Creación de cuenta gratuita (plan Beginner) en `www.pythonanywhere.com` con nombre de usuario 'logicweb'.
9. Compresión del proyecto en un archivo `.zip` para su carga.



10. Carga del archivo .zip mediante la sección Files de PythonAnywhere.
11. Descompresión del archivo en la consola Bash de PythonAnywhere:

```
cd ~  
unzip proyecto_final.zip
```

12. Instalación de Flask en el entorno de PythonAnywhere:

```
pip3 install --user flask
```

13. Creación de la aplicación web desde el menú Web > Add a new web app.
14. Selección de Flask como framework y Python 3.10 como versión.

PASO 9 – Configuración del Archivo WSGI en PythonAnywhere

El archivo WSGI es el punto de entrada de la aplicación en PythonAnywhere. Se editó el archivo generado automáticamente para que apunte correctamente a app.py:

```
import sys  
  
# Ruta al directorio del proyecto  
path = '/home/logicweb/PROYECTO_FINAL'  
if path not in sys.path:  
    sys.path.append(path)  
  
from app import app as application
```

Adicionalmente se configuró el directorio de trabajo (Working directory) y el archivo fuente (Source code) en la interfaz web de PythonAnywhere apuntando a la carpeta del proyecto.

PASO 10 – Inicialización de la Base de Datos en Producción

Una vez configurado el WSGI, se ejecutó la consola Bash de PythonAnywhere para inicializar la base de datos en el entorno de producción:

```
cd ~/PROYECTO_FINAL  
python3 -c "from app import init_db; init_db()"
```

Esto creó el archivo base_datos.db en el directorio del proyecto con las tres tablas vacías listas para recibir datos.

PASO 10 – Verificación y Puesta en Producción

Se recargó la aplicación web desde el panel de PythonAnywhere (botón Reload) y se accedió a la URL pública del sistema: <https://logicweb.pythonanywhere.com>.

Se realizaron las siguientes pruebas finales en el entorno de producción:

- Acceso a la pantalla de login desde un navegador externo.
- Registro de un usuario nuevo ('Alexander Guaman') y verificación de persistencia.
- Navegación por todos los módulos del sistema.
- Resolución de ejercicios y verificación del historial en el módulo de Progreso.
- Prueba desde dispositivos móviles para validar el diseño responsive.

El sistema respondió correctamente en todos los escenarios probados, confirmando el despliegue exitoso del aplicativo web LogicWeb UTA en producción.

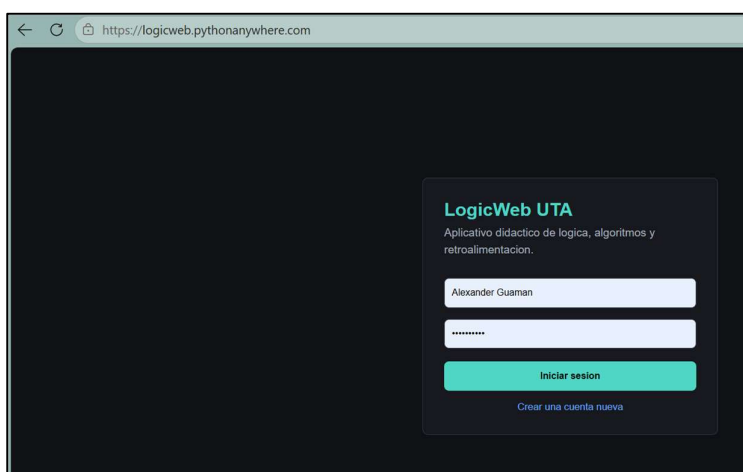


Figura 15. Despliegue del sistema en PythonAnywhere

***Descripción:** La figura evidencia que la aplicación LogicWeb UTA fue publicada correctamente en un entorno de alojamiento web accesible mediante una URL pública.*

Análisis funcional: El despliegue en PythonAnywhere valida que el proyecto no se limita al entorno local de desarrollo, sino que opera en producción bajo configuración WSGI y acceso real por navegador.

Contribución al proyecto: Demuestra la culminación exitosa del ciclo de desarrollo, implementación y disponibilidad pública del sistema.

15.1 Control de versiones y repositorio del proyecto

Como parte del proceso de desarrollo de **LogicWeb UTA**, se utilizó la plataforma **GitHub** como herramienta de control de versiones y alojamiento del código fuente del sistema. El uso de este repositorio permitió organizar de manera estructurada los archivos principales del proyecto, mantener un registro de cambios realizados durante la implementación y facilitar la administración técnica del aplicativo web.



A través del repositorio se almacenaron componentes esenciales del sistema, entre ellos el archivo principal app.py, las plantillas HTML ubicadas en la carpeta templates, el archivo de dependencias del entorno y la documentación general del proyecto. Esta organización permitió conservar la trazabilidad del desarrollo, mejorar la mantenibilidad del código y disponer de una referencia centralizada del producto implementado.

El uso de GitHub también representa una buena práctica en ingeniería de software, ya que favorece la gestión ordenada del código, la documentación del avance del proyecto y la posibilidad de reutilización o ampliación futura del sistema. En el contexto académico, esta herramienta constituye además una evidencia técnica del proceso de construcción del aplicativo.

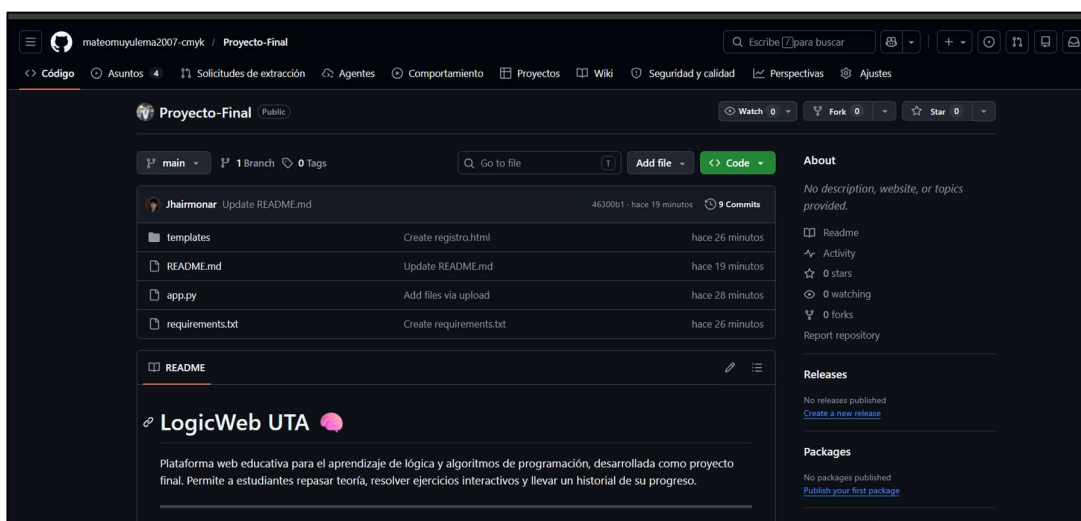


Figura 16. Repositorio del proyecto LogicWeb UTA en GitHub

Descripción: La figura muestra el repositorio del proyecto en GitHub, donde se visualizan los archivos principales del sistema, la estructura general del código fuente y la documentación asociada al desarrollo del aplicativo.

Análisis: Esta evidencia permite verificar la organización técnica del proyecto, el uso de control de versiones y la disponibilidad de los componentes fundamentales del sistema en un entorno de gestión de código.

Contribución al proyecto: Fortalece la trazabilidad, el mantenimiento y la formalización técnica del proceso de desarrollo de LogicWeb UTA.

Enlace del repositorio: <https://github.com/mateomuyulema2007-cmyk/Proyecto-Final>



16. Integración de Conocimientos de la Asignatura

Unidad	Contenidos aplicados	Evidencia en el Proyecto
U1	Lógica, algoritmos, entrada-proceso-salida, pseudocódigo.	Teoría documentada, ejemplos guiados y análisis de casos reales.
U2	Variables, operadores, condicionales, ciclos, contadores y acumuladores.	Formularios de ejercicios con validaciones y cálculos en Python/Flask.
U3	Funciones, encapsulación, modularidad.	Funciones Python en app.py para cada módulo del sistema.
U4	Estructuras de datos básicas.	Listas de prácticas, historial de intentos y tablas SQLite.
MVC	Patrón de diseño de software.	Toda la arquitectura: Vista (HTML), Controlador (Flask), Modelo (SQLite).

El proyecto evidencia una integración directa entre los contenidos de la asignatura y su aplicación práctica en un sistema real. No se trata únicamente de una demostración técnica, sino de un producto que convierte los contenidos académicos en funcionalidades concretas.

17. Metodología de desarrollo y división del trabajo

17.1 Metodología utilizada

El equipo adoptó una **metodología ágil híbrida** que combinó el patrón de diseño **MVC (Modelo-Vista-Controlador)** con un proceso de desarrollo **iterativo e incremental**, inspirado en los principios de Scrum. Este enfoque permitió dividir el trabajo en ciclos cortos y verificables, reducir riesgos técnicos tempranos y mantener siempre una versión funcional del sistema al cierre de cada iteración.

17.2 Planificación del proyecto

Al inicio del proyecto el equipo realizó una reunión de planificación general donde se identificaron los requisitos funcionales, se definió la arquitectura MVC, se diseñó el esquema de base de datos con las cuatro tablas principales (usuarios, practicas, promedios, actividades_modulo) y se distribuyeron los módulos entre los integrantes según sus fortalezas. A



partir de esa reunión se estableció un calendario de cinco sprints de aproximadamente una semana cada uno

17.3 Sprints o iteraciones realizadas

Sprint	Duracion aprox.	Objetivo	Entregable Verificado
Sprint 1	Semana 1-2	Autenticación y estructura base.	Login, registro, logout, sesiones Flask, 4 tablas SQLite
Sprint 2	Semana 3-4	Módulos teóricos básicos	Variables, Condicionales, Bucles, Contadores, Acumuladores con 5 actividades cada uno.
Sprint 3	Semana 5	Módulos avanzados	Funciones, POO Básica, Comparar Lenguajes con actividades evaluadas
Sprint 4	Semana 6	Práctica interactiva.	Ejercicios Interactivos, Laboratorio Lógico, Mis Notas.
Sprint 5	Semana 7-8	Seguimiento y despliegue.	Módulo de Progreso, Panel Reportes Docente, despliegue en PythonAnywhere

17.4 Ceremonias Agiles Aplicadas

Reuniones de planificación de sprint: Al inicio de cada ciclo el equipo definió los módulos o funcionalidades a desarrollar, los criterios de aceptación y quién lideraría cada tarea.

Revisiones de avance: Al cierre de cada sprint se revisó el sistema en funcionamiento en localhost y luego en producción, verificando que cada ruta, actividad evaluada y módulo respondiera correctamente.

Retrospectivas: Al finalizar cada sprint el equipo identificó qué funcionó bien, qué dificultades se presentaron y qué mejoras aplicar al siguiente ciclo.

Coordinación continua: A lo largo del proyecto el equipo mantuvo comunicación constante mediante WhatsApp para reportar bloqueos, compartir fragmentos de código y coordinar la integración de las capas Vista, Controlador y Modelo.



17.5 Division del trabajo entre integrantes

La distribución de responsabilidades se alineó con las capas del patrón MVC y representó el siguiente peso de contribución dentro del equipo:

Jahir Monar (40%): Lideró el desarrollo técnico del proyecto. Fue responsable de la arquitectura MVC completa, incluyendo las rutas Flask (Controlador), la lógica de negocio y acceso a datos en SQLite (Modelo), los módulos de práctica interactiva, el Laboratorio Lógico, el Panel de Reportes Docente y el despliegue en PythonAnywhere. Actuó como referente técnico del equipo en decisiones de integración y arquitectura.

Alexander Guaman (20%): Tuvo a su cargo el desarrollo de la capa Vista, trabajando en las plantillas HTML con Jinja2, el diseño dark mode y la navegación lateral del sistema. Participó en la redacción del informe en las secciones de Marco Teórico, Arquitectura y Requerimientos.

Alejandro Villacres (20%): Se enfocó en el diseño del contenido teórico de los módulos educativos y en la revisión de las actividades evaluadas. Estuvo a cargo de las secciones del informe correspondientes a Cronograma, Resultados, Conclusiones y Recomendaciones.

Mateo Muyulema (20%) Realizó las pruebas funcionales del sistema en cada iteración, verificando los flujos principales antes del despliegue. Contribuyó en la redacción del informe en las secciones de Planteamiento del Problema, Justificación, Objetivos y Anexos.

18. Tabla de Particcipaciones

Nombre	Rol en proyecto	Funcionalidad	Porcentaje contribucion	Descripcion responsabilidades
Jahir Monar	Desarrollador Full-Stack / Arquitecto de Software	Arquitectura MVC completa, rutas Flask, base de datos SQLite, ejercicios interactivos, Laboratorio Lógico, módulo de Progreso, Panel de Reportes Docente, despliegue en PythonAnywhere	40%	Responsable del desarrollo técnico integral del sistema y del despliegue en producción.



Alexander Guaman	Desarrollador Frontend / Documentador	Plantillas HTML, diseño dark mode, sidebar, redacción del informe (Marco Teórico, Arquitectura, Requerimientos)	20%	Desarrollo de la capa Vista y documentación técnica del proyecto.
Alejandro Villacres	Semana Desarrollador de Contenidos / Documentador	Contenido teórico de módulos educativos, revisión de actividades, redacción del informe (Cronograma, Resultados, Conclusiones)	20%	Estructuración del contenido pedagógico y documentación académica.
Mateo Muyulema	Analista / Tester / Documentador	Pruebas funcionales del sistema, redacción del informe (Problema, Justificación, Objetivos, Anexos)	20%	Verificación funcional del sistema y elaboración de secciones del informe.

19. Resultados Obtenidos

Al finalizar el proyecto, se obtuvieron los siguientes resultados concretos y verificables:

- Sistema web completamente funcional y navegable, accesible en <https://logicweb.pythonanywhere.com>.
- Módulo de autenticación operativo: registro, login y logout funcionando correctamente.
- Seis módulos de contenido teórico organizados y desplegados.
- Ejercicios interactivos con retroalimentación automática implementados en cada módulo.
- Persistencia de datos en SQLite: historial de prácticas y promedios almacenados por usuario.
- Módulo de progreso con métricas de efectividad calculadas dinámicamente.
- Interfaz dark mode coherente, profesional y adaptable a distintas resoluciones.
- Código Python documentado con comentarios en cada función y ruta.
- Despliegue en la nube exitoso con disponibilidad 24/7.



Los resultados alcanzados demuestran el cumplimiento de los objetivos planteados y la viabilidad del sistema como herramienta educativa de apoyo.

20. Cronograma de Ejecución

Semana	Actividad principal	Resultado obtenido
1	Definición del problema, objetivos y alcance.	Propuesta aprobada y stack tecnológico definido.
2	Levantamiento de requerimientos y diseño funcional.	Lista de requerimientos y módulos definidos.
3	Diseño de datos, modelo ER y arquitectura Flask.	Esquema SQLite y estructura de app.py definida.
4	Diseño de interfaz y theme dark mode.	HTML/CSS base con sidebar y colores del sistema.
5	Desarrollo del módulo de autenticación.	Login, registro y logout funcionando en local.
6	Desarrollo del dashboard y módulos de contenido.	Seis módulos teóricos navegables.
7	Desarrollo de ejercicios interactivos.	Ejercicios con cálculo y retroalimentación en todos los módulos.
8	Módulo de progreso y persistencia completa.	Historial, métricas y promedios operativos.
9	Pruebas funcionales y despliegue en PythonAnywhere.	Sistema en producción: logicweb.pythonanywhere.com.
10	Documentación final y preparación de la exposición.	Informe completo y presentación académica listos.

21. Recursos Utilizados

21.1 Recursos Tecnológicos

- Computadores portátiles con Windows 10/11 para el desarrollo.
- Visual Studio Code como entorno de desarrollo integrado (IDE).
- Python 3 con Flask, Jinja2 y SQLite3 (librería estándar).



- Navegadores Chrome y Firefox para pruebas.
- PythonAnywhere (plan Beginner gratuito) para el despliegue.
- Git para control de versiones del código fuente.

21.2 Presupuesto

Concepto	Tipo de recurso	Costo
Equipos de cómputo	Propio	\$0 (disponible)
Python, Flask, SQLite	Open Source	\$0
Visual Studio Code	Gratuito	\$0
PythonAnywhere (hosting)	Plan gratuito	\$0
Conectividad a internet	Propia/institucional	Incluida
TOTAL DEL PROYECTO		\$0

El proyecto se ejecutó íntegramente con herramientas gratuitas y open-source, sin costo adicional alguno.

22. Riesgos del Proyecto

Riesgo	Descripción	Medida aplicada
Integración técnica	Dificultades en la conexión entre Flask y SQLite.	Desarrollo modular con pruebas por etapas.
Configuración WSGI	Errores al configurar PythonAnywhere.	Revisión de logs de error y documentación oficial.
Errores lógicos	Resultados incorrectos en ejercicios.	Pruebas unitarias con valores conocidos.
Interfaz poco usable	Dificultad de navegación para el estudiante.	Diseño iterativo con revisiones de usabilidad.

23. Conclusiones

El desarrollo del aplicativo web LogicWeb UTA demostró que es posible construir e implementar un sistema educativo completo y funcional utilizando exclusivamente herramientas open-source dentro del período académico establecido.



La elección de Python/Flask como stack de backend resultó acertada para un proyecto académico: su curva de aprendizaje accesible, la claridad de su código y la facilidad de integración con SQLite y Jinja2 permitieron enfocarse en la lógica del negocio más que en la infraestructura.

El despliegue exitoso en PythonAnywhere evidencia que el sistema trasciende el entorno local y puede ser utilizado por estudiantes reales desde cualquier dispositivo con conexión a internet, cumpliendo con el objetivo de accesibilidad planteado.

El módulo de progreso e historial de prácticas constituye una contribución significativa desde la perspectiva pedagógica, ya que permite al estudiante y al docente visualizar de manera objetiva la evolución del aprendizaje a lo largo del tiempo.

Finalmente, el proyecto integra de manera directa y verificable los contenidos de todas las unidades de la asignatura de Algoritmos de Programación: variables, condicionales, ciclos, contadores, acumuladores y funciones están presentes tanto en el código del sistema como en los ejercicios interactivos para el usuario.

24. Recomendaciones

- Incorporar hashing de contraseñas (bcrypt o werkzeug.security) para mejorar la seguridad del módulo de autenticación.
- Agregar más ejercicios interactivos por módulo para aumentar la variedad de práctica disponible.
- Implementar un módulo de quizzes con opción múltiple para evaluar la comprensión teórica.
- Considerar la migración a PostgreSQL para mayor escalabilidad si el número de usuarios crece.
- Añadir exportación del historial de progreso en formato PDF para reportes académicos.
- Incorporar un módulo de ranking o gamificación para motivar a los estudiantes.

25. Bibliografía/ Documentación web

- Pressman, R. S. (2014). *Ingeniería del software: Un enfoque práctico* (7.^a ed.). McGraw-Hill.
- Sommerville, I. (2011). *Ingeniería de software* (9.^a ed.). Pearson Educación.
- Joyanes Aguilar, L. (2008). *Fundamentos de programación* (4.^a ed.). McGraw-Hill.
- Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed.). O'Reilly Media.
- Pallets. (s.f.). *Flask documentation*. <https://flask.palletsprojects.com>



- SQLite. (s.f.). *SQLite documentation*. <https://www.sqlite.org/docs.html>
- PythonAnywhere. (s.f.). *PythonAnywhere help*. <https://help.pythonanywhere.com>
- MDN Web Docs. (s.f.). *HTML5, CSS3, JavaScript*. <https://developer.mozilla.org>

26. Anexos

Anexo A – Rúbrica de Evaluación

Criterio	Puntaje	Evidencia en LogicWeb UTA
Planteamiento del problema y objetivos	1.0	Problema real documentado, objetivos SMART cumplidos.
Diseño del sistema y arquitectura	1.5	Arquitectura 3 capas, ER, rutas Flask documentadas.
Implementación funcional del aplicativo	3.0	Sistema completo y desplegado en producción.
Aplicación de lógica y algoritmos	2.0	Ejercicios con condicionales, ciclos y validaciones.
Funciones y estructuras de datos	1.0	Funciones Python, tablas SQLite, listas de prácticas.
Calidad didáctica e interfaz	1.0	UI dark mode, retroalimentación y módulo de progreso.
Documentación y defensa	0.5	Informe completo, código comentado, demo en vivo.
TOTAL		

Anexo B – Propuesta de Ejercicios de Vida Real Implementados

- Promedio de notas de un estudiante (3 notas → promedio y estado aprobado/reprobado).
- Cálculo de sueldo de un empleado con horas extras (sueldo base + horas extra × tarifa).
- Determinación de descuentos en una tienda (precio y porcentaje → precio final con descuento).
- Clasificación de edades (edad → categoría: niño, adolescente, adulto, adulto mayor).
- Control de ventas (cantidad vendida × precio unitario → total a cobrar).



-
- Inventario básico (producto, cantidad, precio unitario → valor total del inventario).
 - Consumo de energía (kWh consumidos × tarifa → factura mensual estimada).

Enlace del repositorio: <https://github.com/mateomuyulema2007-cmyk/Proyecto-Final>

Anexo C – URL de Acceso al Sistema

Sistema en producción: <https://logicweb.pythonanywhere.com>

Para acceder al sistema puede registrar un nuevo usuario o utilizar las credenciales de demostración proporcionadas durante la exposición.